

END-TERM PROJECT - 01 & 02 REPORT

ON

CAR DODGING GAME & FLAPPY BIRD GAME

B.Tech. [Computer Science and Engineering]

Semester: 6th

Section: 23412F3

Course Title & Code: Game Programming with C++ (CSE 3545)

Submitted By:

Student Name: Amulya Shrivastava

Registration No: 2341019456

Guided By: Mr. Sougata Sheet

Academic Year: 2025-26



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING AND TECHNOLOGY, INSTITUTE OF
TECHNICAL EDUCATION AND RESEARCH
SIKSHA 'O' ANUSANDHAN (DEEMED TO BE) UNIVERSITY
BHUBANESWAR, ODISHA, INDIA

ABSTRACT

This project presents the development of a **Car Dodging Game** using C++ and the SFML (Simple and Fast Multimedia Library) framework. The game is designed around a player-controlled vehicle that must avoid continuously appearing enemy cars on a moving road while achieving the highest possible score. The implementation demonstrates important concepts of real-time game development such as graphical rendering, keyboard interaction, collision detection, enemy spawning, score tracking, and progressive difficulty adjustment. Object-oriented programming techniques including inheritance, abstraction, and polymorphism were applied through well-structured classes such as PlayerCar and EnemyCar, ensuring modularity and code reusability. Additional features like responsive controls, sound integration, and smooth gameplay mechanics enhanced the overall user experience. The project also involved testing and debugging to maintain stability and performance under different gameplay conditions, providing practical exposure to software design, logical problem-solving, and multimedia application development using C++ and SFML.

This project also includes the creation of a **Flappy Bird** inspired game developed using C++ and SFML, aimed at demonstrating the integration of graphics, animation, and event-driven programming into an interactive gaming application. In the game, the player controls a bird that must pass through moving pipes without collisions, with the score increasing based on successful navigation. The project highlights essential game programming concepts such as game loops, obstacle generation, collision handling, smooth animation, keyboard-based controls, and real-time rendering. File handling techniques were implemented to maintain high scores, while structured class design and efficient memory management improved maintainability and performance. The game serves as a practical example of applying object-oriented programming principles to build engaging multimedia applications and strengthens understanding of graphical user interfaces, user interaction, and dynamic gameplay systems.

Amulya Shrivastava

Mr. Sougata Sheet

Submitted By

Guided By

TABLE OF CONTENTS

Section No.	Topic	Page No.
01	Introduction	04
02	Problem Statement	05
03	Objectives of the Project	06
04	Tools and Technologies Used	07
05	Methodology	08 – 09
06	Results Analysis and Screenshots	10 – 14
07	Logical Understanding	15
08	Conclusion	16
09	Future Scopes and Application	17
	References	18
	Annexure 1	19 - 31
	Annexure 2	32 - 47

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Start Screen For Project -- 1	10
Figure 2	Game Screen For Project - 1	10
Figure 3	Game Screen For Project - 1	11
Figure 4	Game Screen For Project - 1	11
Figure 5	Game Screen For Project - 1	12
Figure 6	Start Screen For Project -- 2	12
Figure 7	Game Screen For Project - 2	13
Figure 8	Game Screen For Project - 2	13
Figure 9	Game Screen For Project - 2	13
Figure 10	Game Screen For Project - 2	14
Figure 11	Game Screen For Project - 2	14

01. Introduction

This project introduces the development of a **Car Dodging Game** using C++ and the SFML library, focusing on the implementation of real-time game programming concepts and interactive multimedia features. C++ was selected because of its high execution speed, efficient memory handling, and strong support for object-oriented programming, which are essential for developing responsive gaming applications. In this game, the player controls a car that moves across lanes to avoid incoming enemy vehicles while the difficulty gradually increases over time. The project incorporates features such as smooth movement, keyboard-based controls, score calculation, collision handling, and dynamic enemy generation to create an engaging gameplay experience. It also provides practical exposure to concepts like event handling, graphics rendering, and modular class design using SFML, while strengthening understanding of software structure, logical thinking, and object-oriented programming principles.

This project also presents the development of a **Flappy Bird** inspired 2D game created using C++ and SFML to demonstrate the integration of graphics, animation, and event-driven programming in a real-time application. The game allows the player to control a bird that must pass through moving obstacles while avoiding collisions, creating an interactive and visually engaging experience. SFML was used to manage graphics rendering, audio support, and user input, ensuring smooth gameplay and responsive controls. The project applies programming concepts such as classes, loops, file handling, and collision detection in a practical environment, while also implementing score management and dynamic obstacle movement. Through this development process, the project improves understanding of game mechanics, user interface design, multimedia programming, and structured software development using object-oriented techniques in C++.

02. Problem Statement

Problem Statement For Project-1 (Car Dodging Game)

The problem addressed in this project is to design and implement a C program that performs a specific task efficiently, build a real-time car dodging game using C++ that satisfies the following requirements:

- Smooth player movement using keyboard controls
- Random enemy car spawning
- Collision detection between player and enemy vehicles
- Score tracking system, Increasing difficulty during gameplay
- Use of Object-Oriented Programming concepts
- Efficient rendering using SFML graphics library

The program is developed to solve a real-world problem using basic programming constructs such as input/output operations, conditional statements, loops, and functions.

The project demonstrates how C language can be used to create simple yet effective applications.

Problem Statement For Project-2 (Flappy Bird Game)

The problem addressed in this project is to design and implement a C program that performs a specific task efficiently, where:

- A bird continuously falls due to gravity.
- The player must control the bird's movement to avoid obstacles (pipes).
- The game ends when a collision occurs.
- The system tracks and stores the highest score achieved.

The program is developed to solve a real-world problem using basic programming constructs such as input/output operations, conditional statements, loops, and functions.

The project demonstrates how C language can be used to create simple yet effective applications

The challenge lies in implementing smooth gameplay, accurate collision detection, and persistent score storage.

03. Objectives of the Project

Objective For Project-1 (Car Dodging Game)

The primary objectives of this project are established to provide a structured approach for the planning, design, and development of the Car Dodging Game while ensuring smooth gameplay, efficient functionality, and effective implementation of game development concepts.

The objectives of this practical project are:

- To develop a functional 2D car dodging game using C++
- To implement Object-Oriented Programming concepts
- To use inheritance and polymorphism through class hierarchy
- To implement collision detection mechanisms
- To create a real-time scoring system
- To improve programming and debugging skills
- To understand game loops and frame-based updates

To gain practical experience using the SFML multimedia library

Objective For Project-2 (Flappy Bird Game)

The objectives of this project are defined to guide the development and implementation of the Flappy Bird Game. They highlight the key features and programming concepts required for the successful completion of the project. The main objectives of this practical project could be as stated.

The objectives of this practical project are:

- To develop a graphical game using C++ and SFML.
- To implement object-oriented concepts such as classes and encapsulation.
- To simulate real-time motion using game loops.
- To implement collision detection logic.
- To maintain and update high scores using file handling.
- To enhance debugging and problem-solving skills.

To gain practical exposure to real-time programming applications.

04. Tools and Technologies Used

The following tools and technologies were used in Project 1 and Project 2:

- Programming Language: C++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
- Graphics and Audio Library: SFML (Simple and Fast Multimedia Library)
- Compiler: g++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
- Development Environment: Visual Studio Code / Terminal-Based Compilation
- Operating System: Ubuntu-24.04.3 LTS

These tools provide the required environment for rendering sprites, handling keyboard input, playing sounds, displaying HUD text, and managing the real-time game loop.

05. Methodology

The methodology followed during the development of Project 1 includes:

1. Requirement analysis:

Identified all essential gameplay mechanics including player movement, enemy vehicle generation, collision detection, score calculation, speed increase, difficulty selection, audio integration, and restart functionality before beginning development.

2. Class design:

Created a base Car class along with derived PlayerCar and EnemyCar classes to implement inheritance and maintain a modular object-oriented structure.

3. Asset loading:

Loaded all required player car, enemy car, font, texture, and sound assets from the Assets folder to support graphics rendering and audio functionality.

4. Game loop:

Implemented the main game loop using SFML window events, delta-time updates, rendering operations, and game state transitions for smooth real-time gameplay.

5. Road rendering:

Designed a vertical road layout containing four lanes with dashed lane markings and scrolling visuals to create a realistic driving environment.

6. Enemy spawning:

Generated enemy vehicles randomly within different lanes and adjusted their movement speed according to the selected difficulty level.

7. Collision and scoring:

Used SFML FloatRect collision boundaries to detect vehicle crashes and updated the score based on player survival time and successfully avoided enemy cars.

Testing:

Tested all major gameplay features including player controls, lane restrictions, collision handling, game-over conditions, pause/resume options, restart functionality, difficulty modes, HUD display, and asset loading performance.

The methodology followed during the development of Project 2 includes:

1. Requirement analysis:

Identified all important game features such as gravity-based movement, bird flapping, obstacle generation, collision detection, score management, high-score storage, difficulty settings, sound effects, and restart functionality.

2. Class design:

Developed a Bird class for player movement and physics, a Pipes class for obstacle generation, and a Game class for handling rendering, scoring, collisions, game states, and asset management.

3. Asset loading:

Loaded all necessary bird sprites, pipe textures, fonts, and audio files from the Assets folder to support smooth graphics and sound integration.

4. Game loop:

Implemented the real-time game loop using SFML events, frame-based updates, rendering functions, and game state management for responsive gameplay.

5. Pipe system:

Created a pipe spawning mechanism where pipe pairs appear from the right side with randomized gap positions and difficulty-dependent spacing.

6. Difficulty system:

Implemented Easy, Medium, and Hard modes with different pipe speeds, gap sizes, and oscillation movements to gradually increase gameplay challenge.

7. Collision and scoring:

Applied SFML FloatRect boundaries to detect collisions with pipes, the top boundary, and the ground while increasing the score whenever the bird successfully crossed a pipe pair.

8. File handling:

Used HighScore.txt file handling techniques to store and retrieve the highest score achieved during gameplay sessions.

06. Results Analysis and Screenshots

Project - 1

The project successfully delivers a fully playable car dodging game with engaging gameplay mechanics and smooth user interaction. It demonstrates the effective implementation of core game development concepts, resulting in an enjoyable and functional gaming experience.

Figure 1: Car Dodging Game Window (Start Window)

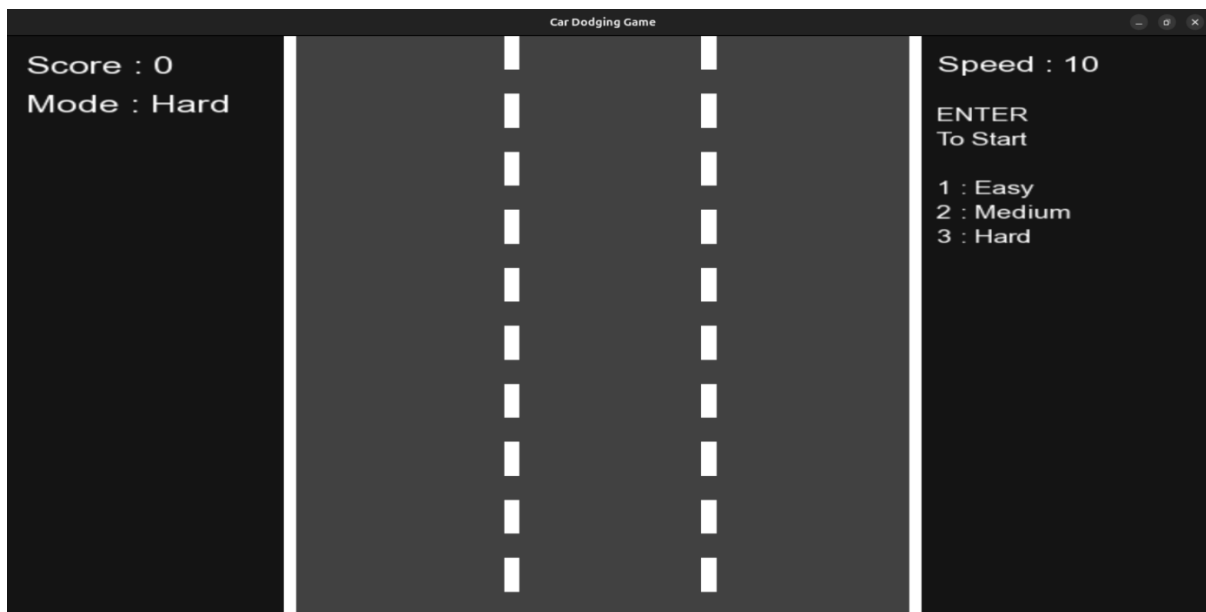


Figure 2: Medium Mode Game Play (By Default Set To Medium)

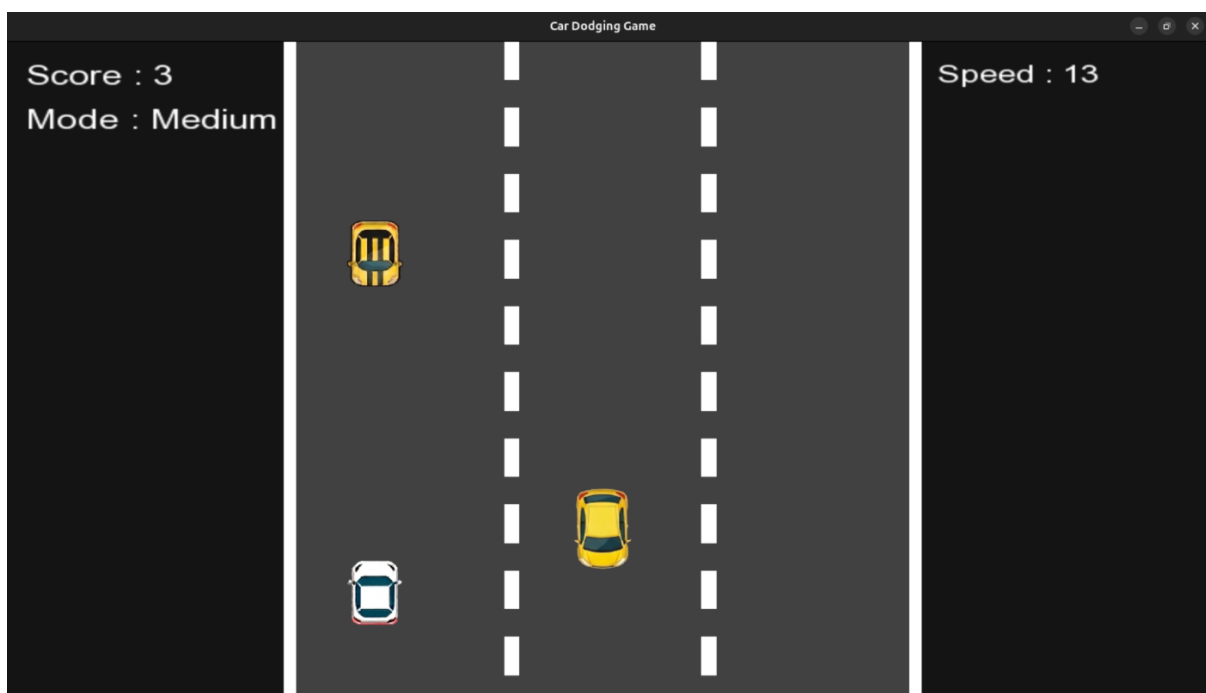


Figure 3: Hard Mode Selected After Restart (Modes Working)

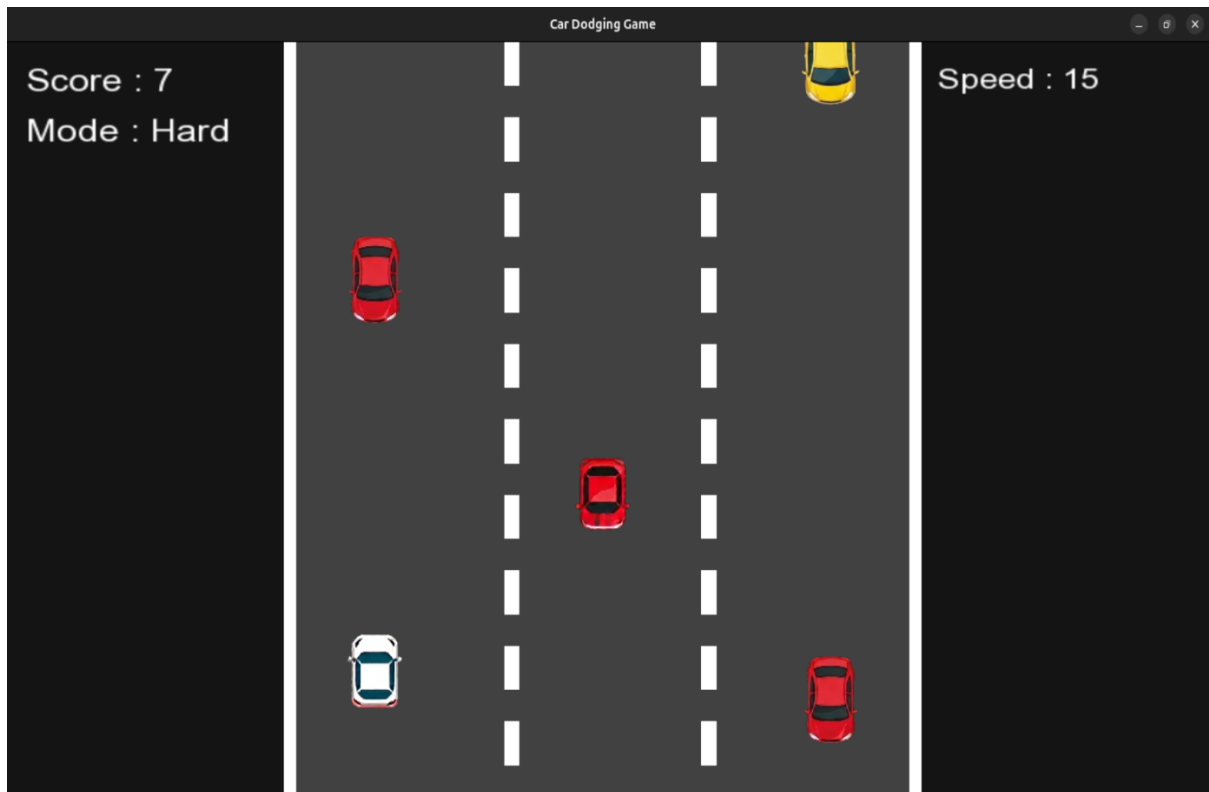


Figure 4: Game Pause and Resume Feature (With P & R Key)

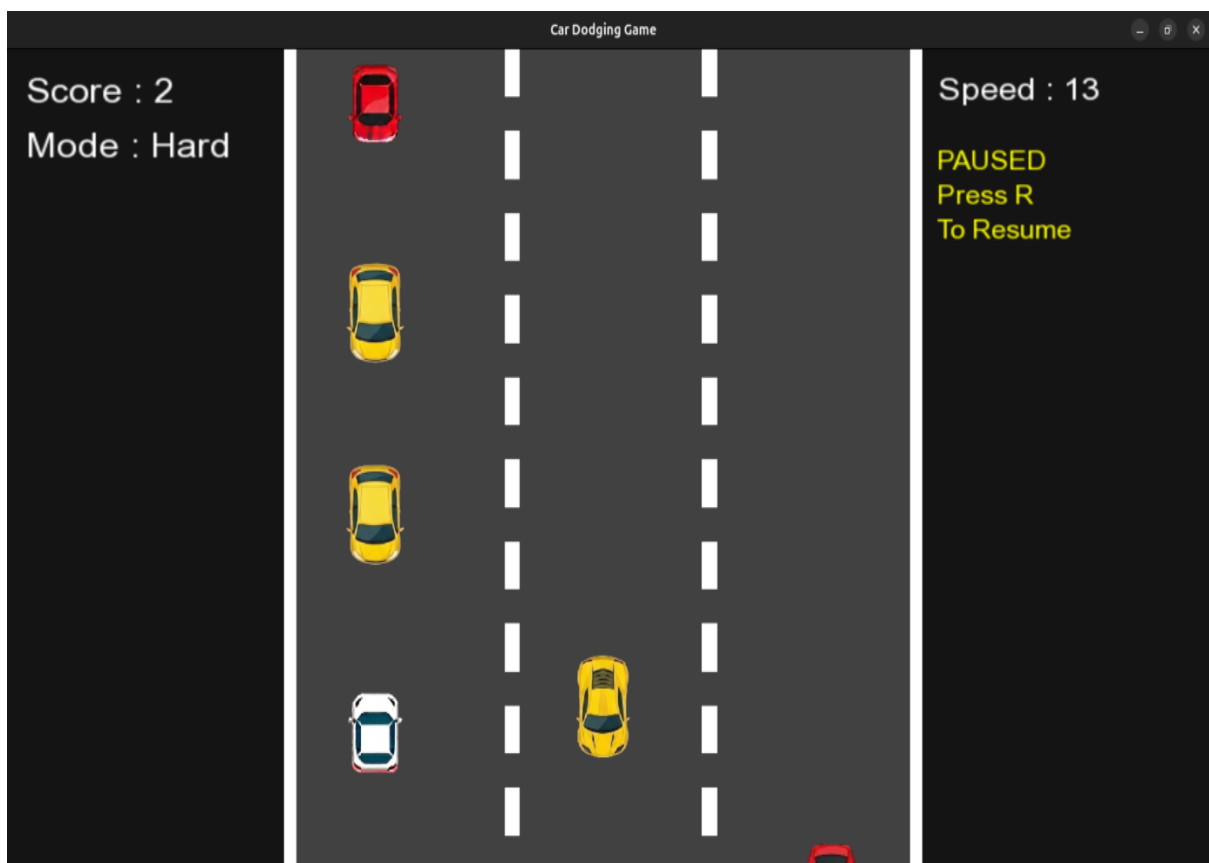
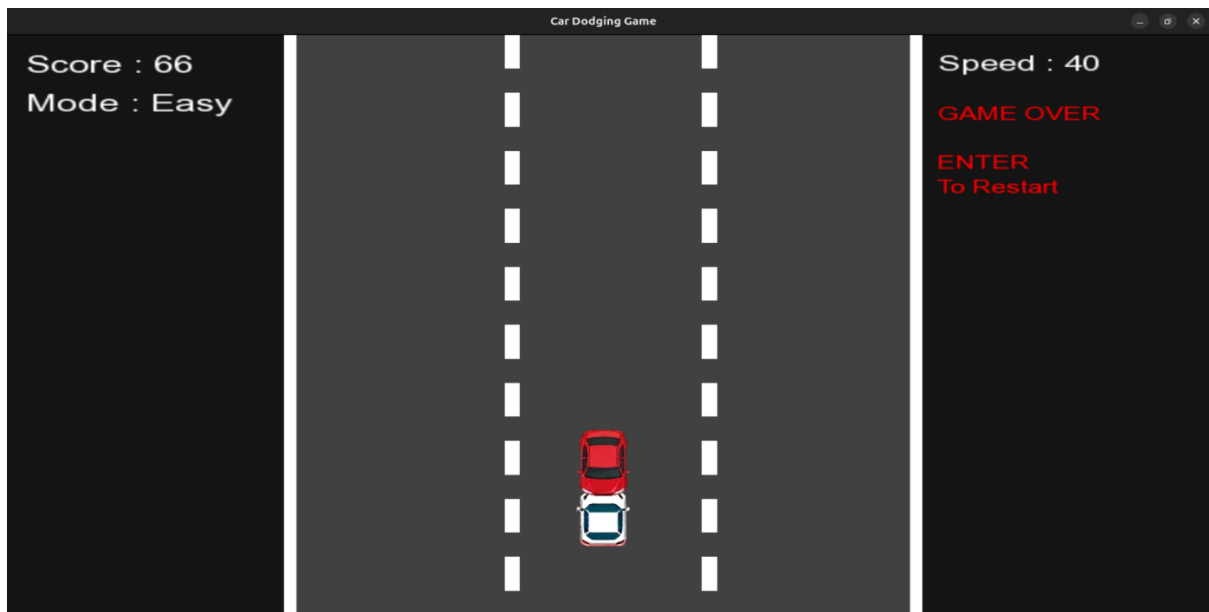


Figure 5: Collision Feature Working And Game Over Happened



Project - 2

The project successfully delivers a fully functional and playable version of the classic Flappy Bird game, featuring smooth gameplay mechanics, responsive controls, and an engaging user experience.

Figure 6: Flappy Bird Game Window (Start Window)



Figure 7: Easy Mode Game Play (Space Is Being Used As Control)

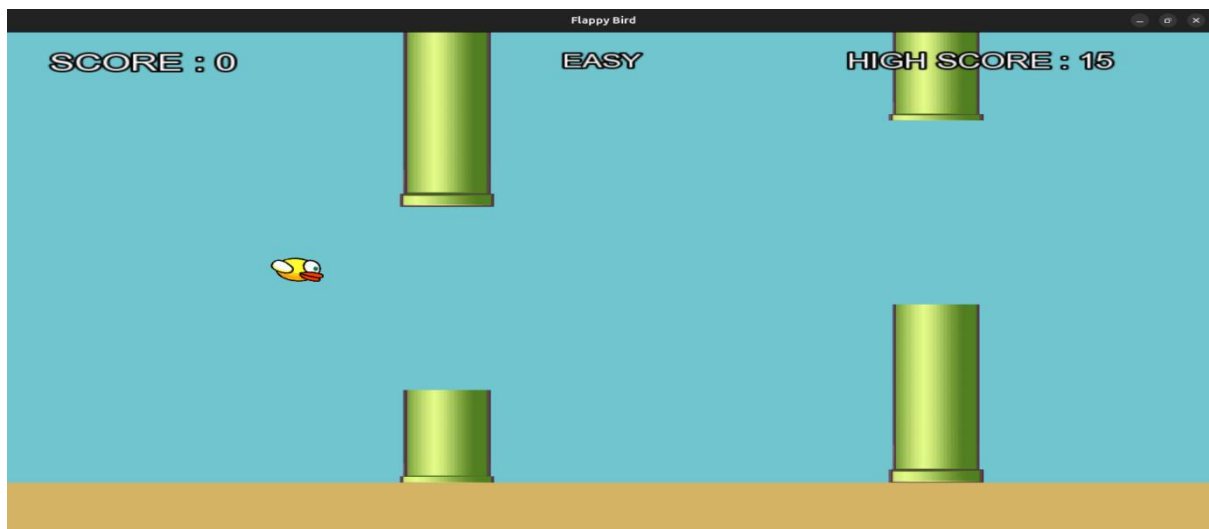


Figure 8: Mode Changed To Medium

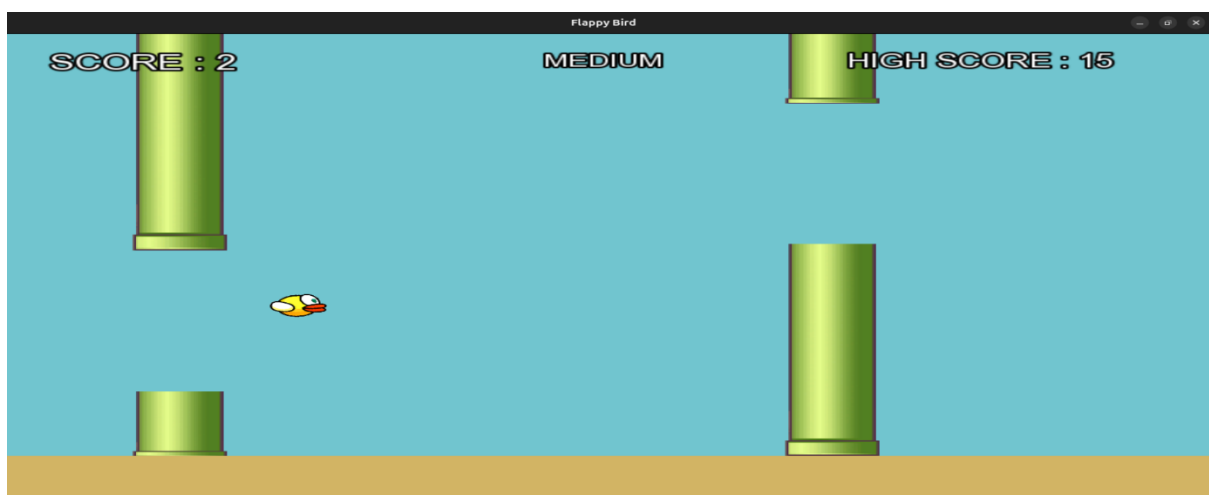


Figure 9: Hard Mode Selected For Game Play



Figure 10: Game Pause And Resume Feature Working (P & R Key Used)

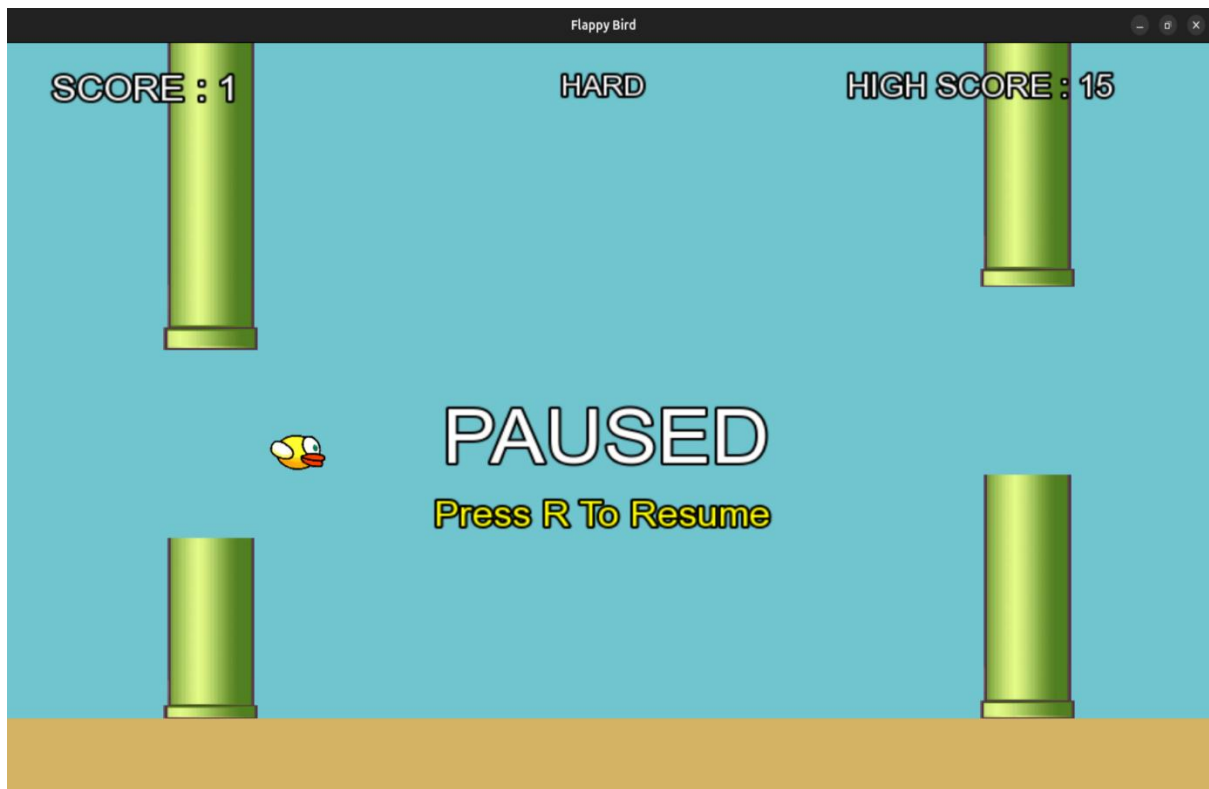
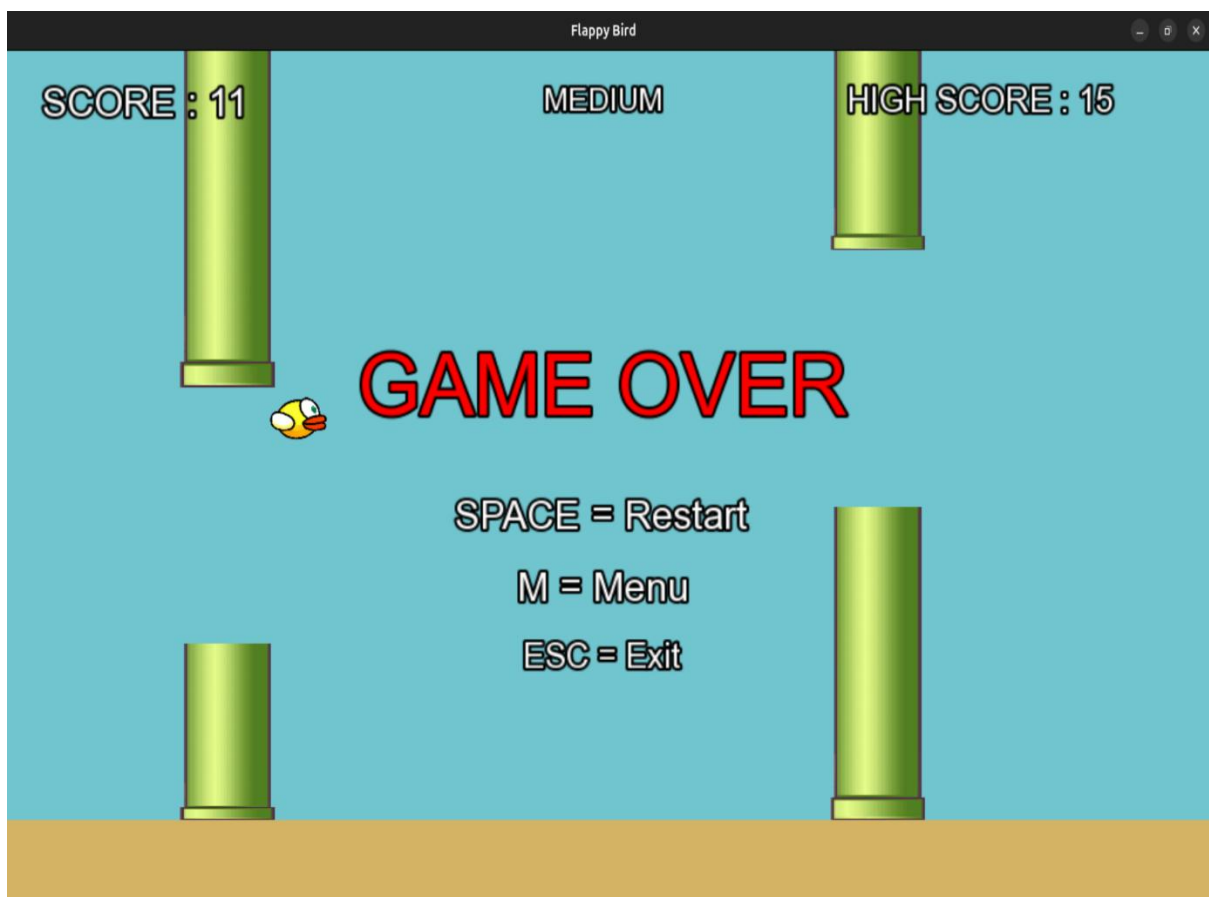


Figure 11: Collision Handled For Bird (Game Over Situation)



07. Logical Understanding

The program flow for **Project-1** is organized around a continuous game execution cycle that manages user input, object movement, collision handling, score updates, and graphical rendering in real time. During each iteration of the loop, the system processes keyboard events, updates the positions of game objects, generates enemy vehicles, and redraws all visual elements on the screen to maintain smooth gameplay.

- The Game class manages the main game window, textures, sounds, score handling, enemy generation, collision checking, and overall game states.
- The Car class acts as a base class containing common properties such as sprite handling, positioning, rendering, and collision boundaries.
- The PlayerCar class extends the Car class and controls player movement across lanes while restricting movement within road limits.
- The EnemyCar class is responsible for enemy vehicle movement, lane allocation, off-screen removal, and score increment when enemies are avoided.
- The update mechanism handles player input, enemy movement, collision verification, difficulty progression, and timed enemy spawning.
- The rendering process displays the road background, player vehicle, enemy cars, score details, and additional screens such as pause menu or game-over interface.

The logical structure for **Project-2** is also based on a real-time game loop where the application continuously updates the bird's movement, obstacle positions, collision status, and score information while rendering the graphics frame by frame. The system processes user input and game events in every cycle to ensure responsive controls and smooth animation throughout the gameplay.

- The Bird class maintains properties related to sprite display, gravity effect, upward flap force, and vertical movement speed.
- The bird update process applies gravity continuously and changes the bird's position according to elapsed frame time.
- The Pipes class manages the generation and movement of upper and lower pipe obstacles along with gap spacing and obstacle speed.
- The pipe update logic shifts obstacles horizontally and can optionally modify their vertical alignment dynamically for increased challenge.
- The Game class controls different states including menu screen, gameplay mode, pause state, and game-over handling.
- The system performs score calculation, collision detection, obstacle spawning, high-score storage using file handling, and audio playback management.
- Collision checking is implemented using optimized boundary calculations to provide balanced and accurate gameplay interaction.

08. Conclusion

In conclusion, the **Car Dodging Game** and **Flappy Bird** style Game were successfully developed using C++ and the SFML library. Both projects demonstrate important real-time game programming concepts such as graphics rendering, event handling, collision detection, scoring systems, and continuous game loop execution. The projects also apply object-oriented programming concepts including inheritance, abstraction, and modular class design, resulting in organized and maintainable code. Through these applications, practical experience was gained in multimedia programming, debugging, and interactive software development.

The Car Dodging Game focuses on an arcade-style environment where the player controls a vehicle and avoids incoming enemy cars on a moving road. Features such as lane-based movement, random enemy generation, increasing speed levels, score tracking, and collision handling were implemented to ensure smooth gameplay. Separate classes for player vehicles, enemy vehicles, and game management improved the structure and efficiency of the application while enhancing understanding of real-time object handling and SFML graphics programming.

The Flappy Bird-inspired game emphasizes gravity-based movement, obstacle avoidance, and side-scrolling gameplay mechanics. The project includes smooth bird movement, dynamically generated pipes, collision detection, score calculation, and high-score storage using file handling. Different game components were managed through separate classes to improve modularity and maintainability. Features such as responsive controls, animations, and sound integration further improved the gameplay experience and strengthened understanding of event-driven programming concepts.

Overall, both projects successfully achieved their objectives and improved knowledge of C++ programming, game logic implementation, multimedia handling, and software design principles. The development process enhanced coding skills, debugging techniques, and logical thinking while providing a strong foundation for future game development projects and advanced interactive applications.

09. Future Scopes and Application

Future Enhancements and Applications for Project-1

The **Car Dodging Game** can be further improved by adding advanced gameplay features and enhanced visual elements to create a more interactive gaming experience. Possible future enhancements include:

- Adding multiple road themes and animated scrolling backgrounds for better visual effects.
- Implementing permanent high-score storage using file handling techniques.
- Introducing power-ups such as shields, speed reduction, and bonus score multipliers.
- Adding different enemy vehicle designs and complex obstacle movement patterns.
- Improving the user interface with interactive menu buttons and better game screens.
- Integrating background music and additional sound effects for movement, collisions, pause, and restart actions.
- Expanding the project into a complete racing or endless-runner style game with advanced levels and gameplay modes.

Future Enhancements and Applications for Project-2

The **Flappy Bird**-style game can also be extended with additional gameplay mechanics and graphical improvements to increase user engagement and overall functionality. Possible future developments include:

- Adding animated bird sprites to create smoother flapping motion.
- Implementing layered scrolling backgrounds for an enhanced side-scrolling effect.
- Replacing fallback audio with dedicated background music and improved sound effects.
- Introducing medals, levels, and dynamic difficulty progression based on player score.
- Adding mouse and touch-based controls for improved accessibility across devices.
- Designing interactive menus with clickable buttons and customizable settings options.
- Creating a leaderboard system that stores high scores along with player names.
- Extending the project into a larger arcade-style side-scrolling game with additional challenges and features.

Both projects can serve as a strong foundation for learning real-time game development, object-oriented programming, multimedia handling, and SFML-based application design while also supporting future expansion into more advanced gaming applications.

References

- (1) Beginning C++ Game Programming by John Horton (Packt)
- (2) Object Oriented Programming with C++ (The McGraw-Hill)
- (3) SFML official documentation: <https://www.sfml-dev.org/documentation/>

Annexure 1 – Car Dodging Game

The following section presents the game source code along with explanations of the different components and functionalities implemented in the project.

Main.cpp:

Serves as the main entry point of the application and manages game initialization, window creation, asset loading, event handling, game loop execution, rendering, collision processing, scoring, difficulty management, and safe program termination.

```
#include <SFML/Graphics.hpp>
#include <SFML/Audio.hpp>
#include <vector>
#include <cstdlib>
#include <algorithm>
#include <string>
#include "PlayerCar.h"
#include "EnemyCar.h"
using namespace sf;

int main()
{
    // Create the game window
    RenderWindow window(VideoMode(800, 600), "Car Dodging Game");
    window.setFramerateLimit(60);

    // Load Textures
    Texture playerTex;

    Texture red1;
    Texture red2;
    Texture yellow1;
    Texture yellow2;
    Texture yellow3;

    // Load Player Car Texture
    if (!playerTex.loadFromFile("Assets/WhiteCar.png"))
    {
        return -1;
    }
    // Load Enemy Car Textures
    if (!red1.loadFromFile("Assets/RedCar1.png"))
    {
        return -1;
    }
    if (!red2.loadFromFile("Assets/RedCar2.png"))
```

```

{
    return -1;
}

if (!yellow1.loadFromFile("Assets/YellowCar1.png"))
{
    return -1;
}

if (!yellow2.loadFromFile("Assets/YellowCar2.png"))
{
    return -1;
}

if (!yellow3.loadFromFile("Assets/YellowCar3.png"))
{
    return -1;
}

// Store enemy textures
std::vector<Texture *> enemyTextures =
{
    &red1,
    &red2,
    &yellow1,
    &yellow2,
    &yellow3};

// Create player object
PlayerCar player(playerTex);

// Set Initial Player Position
player.getSprite().setPosition(550, 500);

// Enemy vector
std::vector<EnemyCar> enemies;

// Score tracker
std::vector<bool> scored;

// Lane Positions
std::vector<float> lanes =
{
    250,
    400,
    550};

Clock clock;
float spawnTimer = 0.0f;
float speedMultiplier = 1.0f;
bool gameOver = false;

```

```

bool paused = false;
bool gameStarted = false;

// Difficulty
int difficulty = 2;
float spawnDelay = 1.0f;
float speedGrowth = 0.05f;

// Load Font
Font font;

if (!font.loadFromFile("Assets/arial.ttf"))
{
    return -1;
}
int score = 0;
// UI Text Objects
Text scoreText;
Text speedText;
Text modeText;
Text gameOverText;
Text pauseText;
Text startText;

// Same Text Size
int uiSize = 24;

// Score Text
scoreText.setFont(font);
scoreText.setCharacterSize(uiSize);
scoreText.setPosition(20, 15);
scoreText.setFillColor(Color::White);

// Mode Text
modeText.setFont(font);
modeText.setCharacterSize(uiSize);
modeText.setPosition(20, 55);
modeText.setFillColor(Color::White);

// Speed Text
speedText.setFont(font);
speedText.setCharacterSize(22);
speedText.setPosition(620, 15);
speedText.setFillColor(Color::White);

// Pause Text
pauseText.setFont(font);
pauseText.setCharacterSize(18);
pauseText.setFillColor(Color::Yellow);

// PERFECTLY ALIGNED WITH SPEED TEXT

```

```

pauseText.setPosition(speedText.getPosition().x, 70);

// Better spacing
pauseText.setLineSpacing(1.2f);

pauseText.setString(
    "PAUSED\n"
    "Press R\n"
    "To Resume");

// Start Menu Text
startText.setFont(font);
startText.setCharacterSize(18);
startText.setFillColor(Color::White);

// PERFECTLY ALIGNED WITH SPEED TEXT
startText.setPosition(speedText.getPosition().x, 70);

// Better spacing
startText.setLineSpacing(1.2f);
startText.setString(
    "ENTER\n"
    "To Start\n\n"
    "1 : Easy\n"
    "2 : Medium\n"
    "3 : Hard");

// Game Over Text
gameOverText.setFont(font);
gameOverText.setCharacterSize(18);
gameOverText.setFillColor(Color::Red);

// PERFECTLY ALIGNED WITH SPEED TEXT
gameOverText.setPosition(speedText.getPosition().x, 70);

// Better spacing
gameOverText.setLineSpacing(1.2f);

gameOverText.setString(
    "GAME OVER\n\n"
    "ENTER\n"
    "To Restart");

// Load Sounds
SoundBuffer crashBuffer;
SoundBuffer moveBuffer;

// Load Crash Sound
if (!crashBuffer.loadFromFile("Assets/death.wav"))
{
    return -1;
}

```

```

}
// Load Move Sound
if (!moveBuffer.loadFromFile("Assets/chop.wav"))
{
    return -1;
}
Sound crashSound;
Sound moveSound;
crashSound.setBuffer(crashBuffer);
moveSound.setBuffer(moveBuffer);
// Background Music
Music bgMusic;

// Load Background Music
if (bgMusic.openFromFile("Assets/bgmusic.wav"))
{
    bgMusic.setLoop(true);
    bgMusic.setVolume(40.f);
    bgMusic.play();
}
// GAME LOOP
while (window.isOpen())
{
    float dt = clock.restart().asSeconds();

    Event event;

    // Event Handling
    while (window.pollEvent(event))
    {
        if (event.type == Event::Closed)
        {
            window.close();
        }

        if (event.type == Event::KeyPressed)
        {
            // Exit Game
            if (event.key.code == Keyboard::Escape)
            {
                window.close();
            }
            // Difficulty Selection
            if (!gameStarted)
            {
                if (event.key.code == Keyboard::Num1 ||
                    event.key.code == Keyboard::Numpad1)
                {
                    difficulty = 1;
                }
            }
        }
    }
}

```



```

    if (event.key.code == Keyboard::Num2 ||
        event.key.code == Keyboard::Numpad2)
    {
        difficulty = 2;
    }
    if (event.key.code == Keyboard::Num3 ||
        event.key.code == Keyboard::Numpad3)
    {
        difficulty = 3;
    }
}
// Start Game
if (event.key.code == Keyboard::Enter &&
    !gameStarted)
{
    gameStarted = true;
}
// Restart Game
if (event.key.code == Keyboard::Enter &&
    gameOver)
{
    enemies.clear();
    scored.clear();
    score = 0;
    spawnTimer = 0;
    speedMultiplier = 1.0f;
    gameOver = false;
    paused = false;
    gameStarted = false;

    // RESET PLAYER COMPLETELY
    player.reset();
}

// Pause
if (event.key.code == Keyboard::P &&
    gameStarted &&
    !gameOver)
{
    paused = true;
}

// Resume
if (event.key.code == Keyboard::R &&
    gameStarted &&
    !gameOver)
{
    paused = false;
}
// PLAYER MOVEMENT FIXED
if (gameStarted &&

```



```

// Spawn Enemy Cars
if (spawnTimer > spawnDelay)
{
    int lane = rand() % 3;
    int texIndex = rand() % enemyTextures.size();
    enemies.push_back(
        EnemyCar(
            *enemyTextures[texIndex],
            lanes[lane],
            lane));
    scored.push_back(false);
    spawnTimer = 0;
}

// Move Enemy Cars
for (auto &e : enemies)
{
    e.move(dt * speedMultiplier);
}

// Score Update
for (size_t i = 0; i < enemies.size(); i++)
{
    if (!scored[i] &&
        enemies[i].getSprite().getPosition().y >
        player.getSprite().getPosition().y)
    {
        score++;
        scored[i] = true;
    }
}

// Collision Detection
for (auto &e : enemies)
{
    if (player.getSprite()
        .getGlobalBounds()
        .intersects(
            e.getSprite()
            .getGlobalBounds()))
    {
        crashSound.play();
        gameOver = true;
        break;
    }
}

// Remove Offscreen Cars
for (size_t i = 0; i < enemies.size(); i++)
{
    if (enemies[i].getSprite().getPosition().y > 700)

```

```

        {
            enemies.erase(enemies.begin() + i);
            scored.erase(scored.begin() + i);
        }
        else
        { i++; }
    }
}
// Update UI Text
scoreText.setString("Score : " + std::to_string(score));

speedText.setString("Speed : " + std::to_string((int)(speedMultiplier * 10)));
std::string diffName;

if (difficulty == 1)
{
    diffName = "Easy";
}
else if (difficulty == 2)
{
    diffName = "Medium";
}
else
{
    diffName = "Hard";
}
modeText.setString("Mode : " + diffName);

// Render
window.clear(Color(20, 20, 20));

// Road
RectangleShape road(Vector2f(420, 600));
road.setFillColor(Color(65, 65, 65));
road.setPosition(190, 0);
window.draw(road);

// Left Border
RectangleShape leftBorder(Vector2f(8, 600));
leftBorder.setFillColor(Color::White);
leftBorder.setPosition(190, 0);
window.draw(leftBorder);

// Right Border
RectangleShape rightBorder(Vector2f(8, 600));
rightBorder.setFillColor(Color::White);
rightBorder.setPosition(602, 0);
window.draw(rightBorder);

// Dotted Road Lines
for (int i = 0; i < 12; i++)

```

```

{
    RectangleShape dash(Vector2f(10, 35));
    dash.setFill(Color::White);

    // Left Divider
    dash.setPosition(335, i * 60);
    window.draw(dash);

    // Right Divider
    dash.setPosition(
        465,
        i * 60);
    window.draw(dash);
}

// Draw Player
if (gameStarted)
{
    window.draw(player.getSprite());
}

// Draw Enemy Cars
for (auto &e : enemies)
{
    window.draw(e.getSprite());
}

// Draw UI
window.draw(scoreText);
window.draw(modeText);
window.draw(speedText);

// Draw Start Menu
if (!gameStarted)
{
    window.draw(startText);
}

// Draw Pause Text
if (paused)
{
    window.draw(pauseText);
}

// Draw Game Over Text
if (gameOver)
{
    window.draw(gameOverText);
}

window.display();
}
return 0;
}

```

Car.h :

Defines the abstract base class for all car objects and provides common functionalities such as sprite handling, positioning, rendering support, and collision boundary access through shared sprite management.

```
#pragma once
#include <SFML/Graphics.hpp>
using namespace sf;

// Abstract Base Class for all cars
class Car
{
protected:
    Sprite sprite;

public:
    // Pure virtual function
    // Must be implemented in child classes
    virtual void move(float dt) = 0;

    // Return sprite reference
    Sprite &getSprite()
    {
        return sprite;
    }
};
```

PlayerCar.h:

Implements the player-controlled car class responsible for lane-based movement, keyboard interaction, initial positioning, sprite scaling, and restricting movement within predefined road boundaries using inherited features from the Car base class.

```
#pragma once
#include "Car.h"
#include <vector>

// Player Car Class
class PlayerCar : public Car
{
private:
    std::vector<float> lanes;
    int currentLane;
    float y;
```

```

public:
    // Constructor
    PlayerCar(Texture &texture)
    {
        sprite.setTexture(texture);

        // Resize player car
        sprite.setScale(0.35f, 0.35f);

        // Set origin to center
        FloatRect bounds = sprite.getLocalBounds();

        sprite.setOrigin(bounds.width / 2, bounds.height / 2);

        // UPDATED ROAD LANES
        lanes = {250, 400, 550};
        currentLane = 2;
        y = 500;

        // Initial Position
        sprite.setPosition(lanes[currentLane], y);
    }

    // RESET PLAYER AFTER RESTART
    void reset()
    {
        currentLane = 2;
        sprite.setPosition(lanes[currentLane], y);
    }

    // Move to left lane
    void moveLeft()
    {
        if (currentLane > 0)
        {
            currentLane--;
            sprite.setPosition(lanes[currentLane], y);
        }
    }

    // Move to right lane
    void moveRight()
    {
        if (currentLane < 2)
        {
            currentLane++;
            sprite.setPosition(lanes[currentLane], y);
        }
    }

    void move(float dt) override
    {
    }
};

```

EnemyCar.h:

Implements the enemy vehicle class responsible for downward movement, lane assignment, enemy spawning above the screen, and movement handling using inherited functionality from the Car base class to support collision detection and dodge-based scoring mechanics.

```
#pragma once
#include "Car.h"

// Enemy Car Class
class EnemyCar : public Car
{
private:
    float speed;
    int laneIndex;

public:
    // Constructor
    EnemyCar(Texture &texture, float x, int lane)
    {
        sprite.setTexture(texture);

        // Resize enemy car
        sprite.setScale(0.35f, 0.35f);

        // Set origin to center
        FloatRect bounds = sprite.getLocalBounds();

        sprite.setOrigin(bounds.width / 2, bounds.height / 2);

        // Spawn above screen
        sprite.setPosition(x, -100);

        speed = 200.0f;
        laneIndex = lane;
    }

    // Move enemy downward
    void move(float dt) override
    {
        sprite.move(0, speed * dt);
    }

    // Return lane index
    int getLane() const
    {
        return laneIndex;
    }
};
```


Annexure 2 – Flappy Bird Game

The following section presents the game source code along with explanations of the different components and functionalities implemented in the project.

Main.cpp:

Acts as the primary entry point of the Flappy Bird game and manages window initialization, asset loading, game states, event handling, bird physics, pipe generation, collision detection, scoring system, high-score file handling, rendering, audio management, and overall gameplay execution.

```
#include <SFML/Graphics.hpp>
#include <SFML/Audio.hpp>
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <ctime>
#include <cstdlib>
#include <algorithm>
#include "Bird.h"
#include "Pipe.h"
using namespace sf;

// Window Settings
const int WIN_W = 900;
const int WIN_H = 600;
const float GROUND_H = 60.f;
const float PIPE_SPAWN_X = WIN_W + 50.f;
const float PIPE_INTERVAL = 2.0f;

// Difficulty Settings
struct DifficultySettings
{
    std::string name;
    float gapSize;
    float pipeSpeed;
    bool oscillate;
};

// EASY / MEDIUM / HARD
const DifficultySettings DIFFICULTIES[3] =
{
    {"EASY", 220.f, 180.f, false},
    {"MEDIUM", 180.f, 240.f, false},
    {"HARD", 150.f, 300.f, true}};
```

```

// Game States
enum class GameState
{
    MENU,
    PLAYING,
    PAUSED,
    GAMEOVER
};

// Load High Score
int loadHighScore()
{
    std::ifstream file("HighScore.txt");
    int hs = 0;
    if (file.is_open())
    {
        std::string text;
        std::getline(file, text);
        std::string prefix = "High Score Till Now Is : ";
        if (text.find(prefix) != std::string::npos)
        {
            std::string number = text.substr(prefix.length());
            hs = std::stoi(number);
        }
        file.close();
    }
    return hs;
}

// Save High Score
void saveHighScore(int hs)
{
    std::ofstream file("HighScore.txt");
    if (file.is_open())
    {
        file << "High Score Till Now Is : " << hs;
        file.close();
    }
}

int main()
{
    std::srand(static_cast<unsigned>(std::time(nullptr)));

    // Create Window
    RenderWindow window(VideoMode(WIN_W, WIN_H), "Flappy Bird");
    window.setFramerateLimit(60);

    // VERY IMPORTANT FOR KEYBOARD INPUT
    window.requestFocus();
}

```

```

// Load Font
Font font;
if (!font.loadFromFile("Assets/arial.ttf"))
{
    std::cout << "Could not load font\n";
}

// Load Textures
Texture birdTex;
Texture pipeTex;

if (!birdTex.loadFromFile("Assets/Bird.png"))
{
    std::cout << "Could not load Bird texture\n";
}

if (!pipeTex.loadFromFile("Assets/Pipes.png"))
{
    std::cout << "Could not load Pipe texture\n";
}

// Ground
RectangleShape ground(Vector2f((float)WIN_W, GROUND_H));

ground.setFillColor(Color(210, 180, 100));
ground.setPosition(0.f, WIN_H - GROUND_H);

// Sounds
SoundBuffer flapBuf;
SoundBuffer hitBuf;
SoundBuffer scoreBuf;

// Correct sound file names
if (!flapBuf.loadFromFile("Assets/chop.wav"))
{
    std::cout << "Could not load flap sound\n";
}

if (!hitBuf.loadFromFile("Assets/death.wav"))
{
    std::cout << "Could not load hit sound\n";
}

if (!scoreBuf.loadFromFile("Assets/out_of_time.wav"))
{
    std::cout << "Could not load score sound\n";
}

Sound flapSound;
Sound hitSound;
Sound scoreSound;

```

```

flapSound.setBuffer(flapBuf);
hitSound.setBuffer(hitBuf);
scoreSound.setBuffer(scoreBuf);

flapSound.setVolume(60.f);
hitSound.setVolume(70.f);
scoreSound.setVolume(50.f);

// Background Music
Music bgMusic;
if (bgMusic.openFromFile("Assets/bgmusic.wav"))
{
    bgMusic.setLoop(true);
    bgMusic.setVolume(40.f);
    bgMusic.play();
}

// Game Variables
GameState state = GameState::MENU;

int diffIndex = 0;
int score = 0;

// Load High Score From File
int highScore = loadHighScore();

Bird bird(birdTex, WIN_W * 0.25f, WIN_H / 2.f);

std::vector<Pipe> pipes;
float pipeTimer = 0.f;

Clock clock;

// Helper Function for Text
auto makeText =
    [&](const std::string &str,
        unsigned size,
        Color color,
        float x,
        float y)
    {
        Text text(str, font, size);
        text.setFillColor(color);
        text.setOutlineColor(Color::Black);
        text.setOutlineThickness(2.f);
        FloatRect r = text.getLocalBounds();
        text.setOrigin(r.width / 2.f, r.height / 2.f);
        text.setPosition(x, y);
        return text;
    };

```

```

// Reset Game
auto resetGame = [&]()
{
    bird.reset(WIN_W * 0.25f, WIN_H / 2.f);
    pipes.clear();
    pipeTimer = 0.f;
    score = 0;
    state = GameState::PLAYING;
};

// GAME LOOP
while (window.isOpen())
{
    float dt = clock.restart().asSeconds();
    if (dt > 0.05f)
        dt = 0.05f;
    Event event;

    // Event Handling
    while (window.pollEvent(event))
    {
        if (event.type == Event::Closed)
        {
            window.close();
        }
    }

    // KEYBOARD INPUT
    if (event.type == Event::KeyPressed)
    {
        // ESC = EXIT GAME
        if (event.key.code == Keyboard::Escape)
        {
            window.close();
        }

        // ----- MENU -----
        if (state == GameState::MENU)
        {
            // EASY
            if (event.key.code == Keyboard::Num1 ||
                event.key.code == Keyboard::Numpad1)
            {
                diffIndex = 0;
                resetGame();
            }

            // MEDIUM
            if (event.key.code == Keyboard::Num2 ||
                event.key.code == Keyboard::Numpad2)
            {
                diffIndex = 1;
            }
        }
    }
}

```

```

        resetGame();
    }

    // HARD
    if (event.key.code == Keyboard::Num3 ||
        event.key.code == Keyboard::Numpad3)
    {
        diffIndex = 2;
        resetGame();
    }
}

// ----- PLAYING -----
else if (state == GameState::PLAYING)
{
    // Bird Jump
    if (event.key.code == Keyboard::Space)
    {
        bird.flap();
        flapSound.play();
    }

    // Pause Game
    if (event.key.code == Keyboard::P)
    {
        state = GameState::PAUSED;
    }
}

// ----- PAUSED -----
else if (state == GameState::PAUSED)
{
    // Resume Game
    if (event.key.code == Keyboard::R)
    {
        state = GameState::PLAYING;
    }
}

// ----- GAME OVER -----
else if (state == GameState::GAMEOVER)
{
    // Restart
    if (event.key.code == Keyboard::Space)
    {
        resetGame();
    }

    // Back To Menu
    if (event.key.code == Keyboard::M)
    {

```

```

        state = GameState::MENU;
    }
}
}

// Game Logic
if (state == GameState::PLAYING)
{
    const DifficultySettings &diff =
        DIFFICULTIES[diffIndex];

    // Bird Physics
    bird.update(dt);

    // Spawn Pipes
    pipeTimer += dt;

    if (pipeTimer >= PIPE_INTERVAL)
    {
        pipeTimer = 0.f;
        pipes.emplace_back(
            pipeTex,
            PIPE_SPAWN_X,
            (float)(WIN_H - GROUND_H),
            diff.gapSize,
            diff.pipeSpeed,
            diff.oscillate);
    }

    // Update Pipes
    for (auto &p : pipes)
    {
        p.update(dt, (float)(WIN_H - GROUND_H));
    }

    // Remove Offscreen Pipes
    pipes.erase(
        std::remove_if(
            pipes.begin(),
            pipes.end(),
            [](const Pipe &p)
            {
                return p.isOffScreen();
            }),
        pipes.end());

    // Score System
    for (auto &p : pipes)
    {
        if (p.checkScore(

```

```

        bird.sprite.getPosition().x))
    {
        score++;
        scoreSound.play();

        // Save New High Score
        if (score > highScore)
        {
            highScore = score;
            saveHighScore(highScore);
        }
    }
}

// Collision Detection
bool hit = false;

for (auto &p : pipes)
{
    if (p.collides(bird.getBounds()))
    {
        hit = true;
        break;
    }
}

// Top or Ground Collision
float birdTop =
    bird.getBounds().top;
float birdBottom =
    bird.getBounds().top +
    bird.getBounds().height;
if (birdTop <= 0.f ||
    birdBottom >= WIN_H - GROUND_H)
{
    hit = true;
}

// GAME OVER
if (hit)
{
    hitSound.play();
    state = GameState::GAMEOVER;
}
}

// DRAWING
window.clear(Color(113, 197, 207));

// Draw Ground
window.draw(ground);

```



```

// ----- MENU -----
if (state == GameState::MENU)
{
    window.draw(makeText(
        "FLAPPY BIRD",
        55,
        Color::Yellow,
        WIN_W / 2.f,
        140.f));

    window.draw(makeText(
        "Press 1 / 2 / 3",
        30,
        Color::White,
        WIN_W / 2.f,
        240.f));

    window.draw(makeText(
        "[1] EASY",
        28,
        Color::Green,
        WIN_W / 2.f,
        320.f));

    window.draw(makeText(
        "[2] MEDIUM",
        28,
        Color::Yellow,
        WIN_W / 2.f,
        380.f));

    window.draw(makeText(
        "[3] HARD",
        28,
        Color::Red,
        WIN_W / 2.f,
        440.f));

    window.draw(makeText(
        "HIGH SCORE : " +
        std::to_string(highScore),
        22,
        Color::White,
        WIN_W / 2.f,
        530.f));
}

// ----- GAMEPLAY -----
else
{

```

```

// Draw Pipes
for (auto &p : pipes)
{
    p.draw(window);
}

// Draw Bird
bird.draw(window);

// Score
window.draw(makeText(
    "SCORE : " +
        std::to_string(score),
    28,
    Color::White,
    110.f,
    30.f));

// Difficulty Name
window.draw(makeText(
    DIFFICULTIES[diffIndex].name,
    22,
    Color::White,
    WIN_W / 2.f,
    30.f));

// High Score
window.draw(makeText(
    "HIGH SCORE : " +
        std::to_string(highScore),
    24,
    Color::White,
    WIN_W - 170.f,
    30.f));

// ----- PAUSED -----
if (state == GameState::PAUSED)
{
    window.draw(makeText(
        "PAUSED",
        60,
        Color::White,
        WIN_W / 2.f,
        WIN_H / 2.f));

    window.draw(makeText(
        "Press R To Resume",
        28,
        Color::Yellow,
        WIN_W / 2.f,
        WIN_H / 2.f + 70.f));
}

```

```

    }

    // ----- GAME OVER -----
    if (state == GameState::GAMEOVER)
    {
        window.draw(makeText(
            "GAME OVER",
            60,
            Color::Red,
            WIN_W / 2.f,
            220.f));

        window.draw(makeText(
            "SPACE = Restart",
            28,
            Color::White,
            WIN_W / 2.f,
            320.f));

        window.draw(makeText(
            "M = Menu",
            28,
            Color::White,
            WIN_W / 2.f,
            370.f));

        window.draw(makeText(
            "ESC = Exit",
            24,
            Color::White,
            WIN_W / 2.f,
            420.f));
    }
}
window.display();
}
return 0;
}

```

Bird.h :

Defines the bird class responsible for sprite management, gravity-based movement, flap control, rotation animation, collision boundary handling, rendering, and position reset functionality for smooth real-time gameplay.

```
#pragma once
#include <SFML/Graphics.hpp>

class Bird
{
public:

    sf::Sprite sprite;
    float velocityY;
    float gravity;
    float flapStrength;

    static const float RENDER_W;
    static const float RENDER_H;

    Bird(sf::Texture &texture,
        float startX,
        float startY)

    {
        velocityY = 0.f;

        // FIXED VALUES
        gravity = 700.f;

        flapStrength = -320.f;
        sprite.setTexture(texture);

        sf::Vector2u texSize = texture.getSize();

        float scaleX = RENDER_W / texSize.x;
        float scaleY = RENDER_H / texSize.y;

        sprite.setScale(scaleX, scaleY);
        sprite.setOrigin(texSize.x / 2.f, texSize.y / 2.f);
        sprite.setPosition(startX, startY);
    }

    // Bird Jump
    void flap()
    {
        velocityY = flapStrength;
    }
}
```

```

// Update Bird
void update(float dt)
{
    velocityY += gravity * dt;
    sprite.move(
        0.f,
        velocityY * dt);

    // Rotation Animation
    float angle = velocityY * 0.05f;

    if (angle > 75.f)
        angle = 75.f;

    if (angle < -25.f)
        angle = -25.f;
    sprite.setRotation(angle);
}

// Collision Bounds
sf::FloatRect getBounds() const
{
    return sprite.getGlobalBounds();
}

// Draw Bird
void draw(sf::RenderWindow &window)
{
    window.draw(sprite);
}

// Reset Bird
void reset(float x, float y)
{
    sprite.setPosition(x, y);
    velocityY = 0.f;
    sprite.setRotation(0.f);
}
};

const float Bird::RENDER_W = 50.f;
const float Bird::RENDER_H = 40.f;

```

Pipe.h:

Implements the pipe obstacle class responsible for generating pipe pairs, handling horizontal movement, random gap positioning, vertical oscillation in hard mode, off-screen removal, score tracking, collision detection, and rendering of obstacles during gameplay.

```
#pragma once
#include <SFML/Graphics.hpp>
#include <cstdlib>
#include <cmath>

// Pipe Class
class Pipe
{
public:
    sf::RectangleShape topRect;
    sf::RectangleShape botRect;
    float x;
    float gapY;
    float gapSize;
    float speed;
    bool scored;
    bool oscillate;
    float oscillateTimer;
    float oscillateAmp;
    float oscillateSpeed;

    static const float PIPE_W;

    // Constructor
    Pipe(sf::Texture &pipeTexture,
        float startX,
        float playAreaH,
        float gap,
        float scrollSpeed,
        bool hardMode = false)
    {
        x = startX;
        gapSize = gap;
        speed = scrollSpeed;
        scored = false;
        oscillate = hardMode;
        oscillateTimer = 0.f;
        oscillateAmp = 55.f;
        oscillateSpeed = 2.0f;

        // Random gap position
        float margin = gapSize / 2.f + 70.f;
        float range = playAreaH - margin * 2.f;
```

```

    if (range < 1.f)
        range = 1.f;
    gapY =
        margin +
        (float)(std::rand() % (int)range);
    topRect.setTexture(&pipeTexture);
    botRect.setTexture(&pipeTexture);
    updateRects(playAreaH);
}

// Update pipe sizes
void updateRects(float playAreaH)
{
    float gapTop = gapY - gapSize / 2.f;
    float gapBottom = gapY + gapSize / 2.f;

    // Top Pipe
    float topH = gapTop;

    if (topH < 0.f)
        topH = 0.f;

    topRect.setSize(sf::Vector2f(PIPE_W, topH));
    topRect.setPosition(x - PIPE_W / 2.f, 0.f);

    // Bottom Pipe
    float botH =
        playAreaH - gapBottom;

    if (botH < 0.f)
        botH = 0.f;

    botRect.setSize(sf::Vector2f(PIPE_W, botH));

    botRect.setPosition(x - PIPE_W / 2.f, gapBottom);
}

// Update movement
void update(float dt, float playAreaH)
{
    // Move left
    x -= speed * dt;

    // Hard mode oscillation
    if (oscillate)
    {
        oscillateTimer += dt;
        float delta =
            oscillateAmp *
            std::sin(
                oscillateSpeed *

```

```

        oscillateTimer) *
        dt;
    gapY += delta;
    float margin = gapSize / 2.f + 70.f;

    if (gapY < margin)
        gapY = margin;

    if (gapY > playAreaH - margin)
        gapY = playAreaH - margin;
}
updateRects(playAreaH);
}

// Remove pipe off screen
bool isOffScreen() const
{
    return (x + PIPE_W / 2.f) < 0.f;
}

// Score check
bool checkScore(float birdX)
{
    if (!scored && birdX > x)
    {
        scored = true;
        return true;
    }
    return false;
}

// Collision check
bool collides(sf::FloatRect birdBounds) const
{
    return birdBounds.intersects(topRect.getGlobalBounds()) ||
        birdBounds.intersects(botRect.getGlobalBounds());
}

// Draw pipes
void draw(sf::RenderWindow &window)
{
    window.draw(topRect);
    window.draw(botRect);
}
};

// Static Width
const float Pipe::PIPE_W = 70.f;

```